

ASSIGNMENT-2

Problem Statement Write a C program for sparse matrix realization and operations on it - Transpose, Fast transpose.

Objective

- 1) To study the concept of sparse matrix, how it is stored & displayed.
- 2) To understand the implementation of sparse matrix operations: Simple transpose, Fast transpose

Implementation:

Platform

- 64 bit Open source Linux or its derivatives
- Open source C programming tool like gcc / eclipse editor

Pseudo code:

```

Step 1 Start
Step 2 Declare variables in int main()
Step 3 Accept values
Step 4 Form a compact matrix
compact(A, m, n, B)
    B(0, 0) = m;
    B(0, 1) = n;
    k = 1;
    for (i = 0 to m)
    {
        for (j = 0 to n)
        {
            if A(i, j) != 0
            {
                B(k, 0) = i;
                B(k, 1) = j;
                B(k, 2) = A(i, j);
                k++;
            }
        }
    }
  
```


$B(0,2) = k-1;$
 $\}$

Step 5

~~Addition(a, b, c)~~

~~if $a(0,0) = b(0,0)$ and $(0,1) = b(0,1);$~~

simple_transpose($\frac{A}{a}, B$)

$\{(m,n,t) := (A(0,0), A(0,1), A(0,2))$

$(B(0,0), B(0,1), B(0,2)) := (n,m,t)$

if $(t \leq 0)$ then return

$\{ q := 1$

for $col := 0$ to n do

for $p := 1$ to t do

if $A(p,1) = col$ then

$\{(B(q,0), B(q,1), B(q,2)) := (A(p,1), A(p,0), A(p,2))$

$q := q+1$

end

end

end

$\} \}$

Step 6

~~fast_transpose($\frac{A}{a}, B$)~~

$\{$

$(m,n,t) := (A(0,0), A(0,1), A(0,2))$

$(B(0,0), B(0,1), B(0,2)) := (n,m,t)$

if $t \leq 0$ then return

for $i := 0$ to n do $S(i) := 0$ end

for $i := 1$ to t do

$S(A(i,1)) := S(A(i,1)) + 1$

end

$T(0) := 1$

for $i := 1$ to n do


```

T(i) := T(i-1) + S(i-1)
end
for (i := 1) to n do
    j := A(i, 1)
    (B(T(j), 0), B(T(j), 1), B(T(j), 2)) := (A(i, 1), A(i, 0), A(i, 2))
    T(j) := T(j) + 1
end
}

```

Step 6

Display all formed matrices

Step 7

Stop.

Time

Complexity

To calculate the time complexity for simple transpose.

$$\text{Total operations} = c \cdot n$$

$$\text{i.e., } O(c \cdot n)$$

$$\text{time complexity} =$$

To calculate the time complexity for fast transpose.

$$\text{Total operations} = c + n + c + n$$

$$= 2(c + n)$$

$$\text{i.e., } O(c + n)$$

$$\text{time complexity} =$$



Conclusion Thus, implemented sparse matrix operations assignment. This system is able to perform different operations on sparse matrices such as simple and fast transpose and their time complexities.

FAQs A sparse matrix is a two-dimensional array or matrix in which the vast majority of the elements have a value of zero. The applications of sparse matrix

1) What is a sparse matrix? List the applications.

A sparse matrix is a two dimensional array matrix in which the vast majority of the elements have a value of zero. The applications of a sparse matrix are:

- Machine Learning and Data Science.
- Network Analysis and Graph Theory
- Scientific computing and engineering.

2) Represent sparse matrices with suitable data structures. Explain with an example simple and fast transpose.

Sparse matrices are generally represented using

- Triplet Representation, and
- Linked List Representation

Eg: Let matrix
$$\begin{bmatrix} 1 & 0 & 3 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$



compact

	col 0	col 1	col 2
row 0	3	3	3
row 1	0	0	1
row 2	0	2	3
row 3	1	2	1
row 4			

Simple transpose

	col 0	col 1	col 2
row 0	3	3	3
row 1	0	0	1
row 2	2	0	3
row 3	2	1	1

Fast transpose

	0	1	2
total	1	0	2
location index			

	col 0	col 1	col 2
row 0	3	3	3
row 1	0	0	1
row 2	2	0	3
row 3	2	1	1



- 3) Find out the addition of two sparse matrices in triplet form and also find simple and fast transpose?

M_1 :

4	5	6
0	3	5
1	3	8
1	4	843
2	3	45
3	2	45
4	1	2

M_2

4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	0	4 21

$M_1 + M_2$

4	5	9
0	3	12
0	4	6
1	3	8
1	4	49
2	1	8
2	3	4
3	2	90
4	1	12
4	4	21

Transpose



Simple transpose

4	5	9
1	2	8
1	4	12
2	3	90
3	0	12
3	1	8
3	2	4
4	0	49 6
4	1	49
4	4	21

Fast transpose

	0	1	2	3	4
total	2	2	2	3	4
index	0	0	2	3	

0	1	2	3	4
0	2	1	3	3

~~23~~
23/7/26



Code:

```
#include <stdio.h>
void accept(int ma[50][50], int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("Enter element [%d][%d]: ", i, j);
            scanf("%d", &ma[i][j]);
        }
    }
}
void display(int ma[50][50], int m, int n)
{
    int i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("%d\t", ma[i][j]);
        }
        printf("\n");
    }
}
void compact(int ma[50][50], int mb[50][50], int m, int n)
{
    int i, j, k = 1;

    mb[0][0] = m;
    mb[0][1] = n;
    mb[0][2] = 0;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (ma[i][j] != 0)
            {
                mb[k][0] = i;
                mb[k][1] = j;
                mb[k][2] = ma[i][j];
                k++;
            }
        }
    }

    mb[0][2] = k - 1;
}
void simple_transpose(int mb[50][50], int mb1[50][50])
{
    int i, j, k = 1;

    mb1[0][0] = mb[0][1];
    mb1[0][1] = mb[0][0];
    mb1[0][2] = mb[0][2];

    for (i = 0; i < mb[0][1]; i++)
    {
        for (j = 1; j <= mb[0][2]; j++)
        {
            if (mb[j][1] == i)
            {
                mb1[k][0] = mb[j][1];
                mb1[k][1] = mb[j][0];
                mb1[k][2] = mb[j][2];
                k++;
            }
        }
    }
}
```

```
void fast_transpose(int mb[50][50], int mb1[50][50])
{
    int total[50] = {0};
    int index[50];
    int i, j;

    mb1[0][0] = mb[0][1];
    mb1[0][1] = mb[0][0];
    mb1[0][2] = mb[0][2];
    for (i = 1; i <= mb[0][2]; i++)
    {
        total[mb[i][1]]++;
    }
    index[0] = 1;
    for (i = 1; i < mb[0][1]; i++)
    {
        index[i] = index[i - 1] + total[i - 1];
    }

    /* Perform fast transpose */
    for (i = 1; i <= mb[0][2]; i++)
    {
        j = index[mb[i][1]];

        mb1[j][0] = mb[i][1];
        mb1[j][1] = mb[i][0];
        mb1[j][2] = mb[i][2];

        index[mb[i][1]]++;
    }
}

int main()
{
    int ma[50][50];
    int mb[50][50];
    int mb1[50][50];
    int m, n, choice;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    printf("\nEnter the matrix:\n");
    accept(ma, m, n);
    printf("\nOriginal Matrix:\n");
    display(ma, m, n);
    compact(ma, mb, m, n);
    printf("\nSparse Matrix (Triplet Form):\n");
    display(mb, mb[0][2] + 1, 3);
    printf("\nChoose Transpose Method:\n");
    printf("1. Simple Transpose\n");
    printf("2. Fast Transpose\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            simple_transpose(mb, mb1);
            printf("\nSimple Transpose:\n");
            display(mb1, mb1[0][2] + 1, 3);
            break;

        case 2:
            fast_transpose(mb, mb1);
            printf("\nFast Transpose:\n");
            display(mb1, mb1[0][2] + 1, 3);
            break;

        default:
            printf("Invalid choice.\n");
    }

    return 0;
}
```


Output:

```
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ gcc 1691assignment2.c
riamudhole@Ubuntu:~/Desktop/DS Sem 3$ ./a.out
Enter number of rows and columns: 2
2

Enter the matrix:
Enter element [0][0]: 1
Enter element [0][1]: 3
Enter element [1][0]: 0
Enter element [1][1]: 0

Original Matrix:
1      3
0      0

Sparse Matrix (Triplet Form):
2      2      2
0      0      1
0      1      3

Choose Transpose Method:
1. Simple Transpose
2. Fast Transpose
Enter your choice: 2

Fast Transpose:
2      2      2
0      0      1
1      0      3
riamudhole@Ubuntu:~/Desktop/DS Sem 3$
```